UNIT-2

DIVIDE AND CONQUER

Iterative Algorithm:

Iterative algorithms are controlled by a loop.

In an iterative approach a function repeats until the condition fails.

Eg: The algorithm for computing sum of n numbers by using iterative method

```
Algorithm sum(n)
//computes the sum iteratively
//input: Integer n
//output: Sum of n numbers
Begin
    Sum=0
    for  i=1 to n do
        sum=sum+i
    end for
end
```

Recursive Algorithm:

In a recursive algorithm, the function calls itself until the condition is met.

The recursive algorithm consists of two cases:

1.Base case

2.Recursive case

Eg: The algorithm for computing sum of n numbers by using recursive method

```
Algorithm sum(n)
//compute sum using recursive method
```

```
//input: Integer n

//output: sum of n numbers

Begin

    If(n equals 0) then

        return 0

  else

       return n+sum(n-1)

 end if

end
```

differences between recursive and iterative methods:

Criteria	Recursion	Iteration
Termination	It terminates when the base condition is reached.	It terminates when the condition is failed.
Space	Extra space in the form of stack space is required for recursion.	No additional space is required.
Stack overflow	Repeated calls requires more space and may cause stack overflow.	No such situation occurs.
Formulation	Recursive programs are easier to formulate.	Formulation of iterative programs is not as easy as recursive programs.
Compactness	Recursive programs are simple and compact.	Iterative programs are not as compact as recursive programs.
Time complexity	Very high (generally exponential) time complexity.	Relatively lower time complexity (generally polynomial-logarithmic).

Divide and Conquer method:

Divide and conquer method work according to the following:

The problem's instance is divided into smaller instances of the same problem ideally of about the same size.

The smaller instances are solved typically, recursively or sometimes through a different algorithm is employed when the instances are small enough.

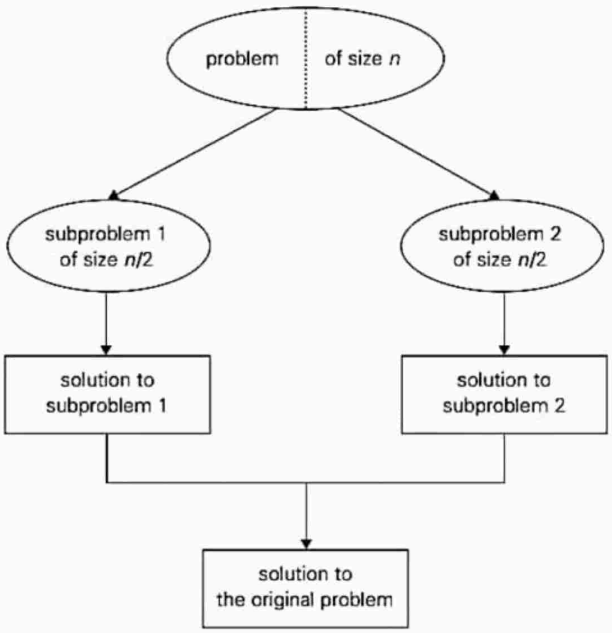
The solutions obtained for each smaller instances are combined to get a solution to the original problem.

Divide and conquer method consists of 3 steps:

Divide: Divide the original problem into subproblems.

Conquer: Solve every subproblem individually.

Combine: Put together the solutions of the sub problems to get solution to the whole problem.



Eg: Let us consider, let us consider the problem of computing the sum of n numbers a0....an-1. If n > 1, we can divide the problem into two instances of

the same problem: to compute the sum of the first [n/2] numbers and to compute the sum problem into compute the sum of the first [n/2] numbers and to compute the sum of the remaining [n/2] numbers. Once each of these two sums is computed by applying the same method recursively, we can add their values to get the sum in question:

a0+....+an-1=(a0+.....+a([n/2]-1)+(a[n/2]+.....+a[n-1])

The following is the recurrence relation for calculating running time T(n):

$$T(n)=a.T(\frac{n}{b})+f(n)$$

Where a - number of partitions

n/b -size of each partition

f(n) - dividing and combining time

Addition of n numbers:

$$T(n)=a.T(\frac{n}{b})+f(n)$$

Where a=2, b=2, f(n)=1

$$T(n)=2T(\frac{n}{2})+1$$

Assuming n= 2^k and applying smoothness rule;

$$T(2^k) = 2T(\frac{2^k}{2})+1$$

$$= 2T(2^{k-1})+1$$

$$= 2[\ 2T(\frac{2^{k-1}}{2})+1] +1$$

$$= 2^2\ [2T(\frac{2^{k-2}}{2})+1]+2+1$$

$$= 2^3\ T(2^{k-3})+4+2+1$$

$$T(2^k)= 2^i\ T(2^{k-i})+ 2^{i-1} + 2^{i-2} + \ldots +2+1$$

Put i=k

$T(2^k) = 2^{k-1} + 2^{k-2} + ...+2+1$ [here T(0)]

By geometric progression;

$$T(2^k) = \frac{2^k-1}{2-1}$$

$$= 2^k - 1$$

[$n = 2^k \Rightarrow k = \log_2 n$]

$$T(2^k) = 2^{\log_2 n} - 1$$

$$= n^{\log_2 2} - 1 = n-1 \in O(n)$$

Master theorem:

If $f(n) \in o(n^d)$ where $d \geq 0$ in a recurrence equation then

$$T(n) \in \begin{cases} o(n^d) & \text{if } a < b^d \\ o(n^d \log n) & \text{if } a = b^d \\ o(n^{\log_b a}) & \text{if } a > b^d \end{cases}$$

Eg : If a=2 b=2 and d=0 then $a > b^d$

Therefore $T(n) \in o(n^{\log_b a}) = o(n)$

Binary search:

Consider a[0...n] is the list of elements and x is the search key. For performing binary search for a sorted list follow the steps below:

1.Find out the middle element of the given sorted list by using:

$$mid = \frac{start + last}{2}$$

Middle element ⇒ a[mid]

2.Compare x with the middle element.

3.If x matches with the middle element, we return the mid index.

4.Else If x is greater than the mid element, then x can only lie in the right half subarray after the mid element. So we search in the right half.

5.Else (x is smaller) we search in the left half.

Eg :Elements are 3,14,27,31,39,42,55,70,74,81,85,93,98 search the value 70 using binary search.

Index	0	1	2	3	4	5	6	7	8	9	10	11	12
Value	3	14	27	31	39	42	55	70	74	81	85	93	98
I-1	l						m						r
I-2								l		m			r
I-3								l	m	r			

Iteration-1:

l=0 ,r=12

m= $\frac{l+r}{2}$ = $\frac{0+12}{2}$ =6

a[6]=55 , k=70

k > a[6]

∴ l=m+1=7,r=12

Iteration-2:

m= $\frac{7+12}{2}$ =9.5 ≈ 9

a[9]=81,k=70

k < a[9]

∴l=7,r=9-1=8

Iteration-3:

m= $\frac{7+8}{2}$ =7.5 $\approx$ 7

a[7]=70 , k=70

k=a[7]

$\therefore$ Element found at position 7 in the given list.

Algorithm for binary search:

Algorithm binarysearch(a[0...n-1])

//algorithm for binary search.

//input: an array A[0...n-1] sorted in ascending order and a search key k.

//output: an idea of array's element that is equal to k or -1 if there is no such element.

l $\leftarrow$ 0 , r $\leftarrow$ n-1

While l $\leq$ r do

m $\leftarrow$ ($\frac{[l+r]}{2}$)

if k=a[m]

return m

else if k < a[m]

r $\leftarrow$ m-1

else

l $\leftarrow$ m+1

return (-1)

Time complexity:

The number of key comparisions required in a worst case is:

C_w (n)= C_w ($\frac{n}{2}$)+1

for n>1 C_w (1)=1

Here a=1,b=2,o(n^d)=1 ⇒ d=0

C_w (n) ε o(n^d log n)

C_w (n) ε o(log n)

Finding minimum and maximum:

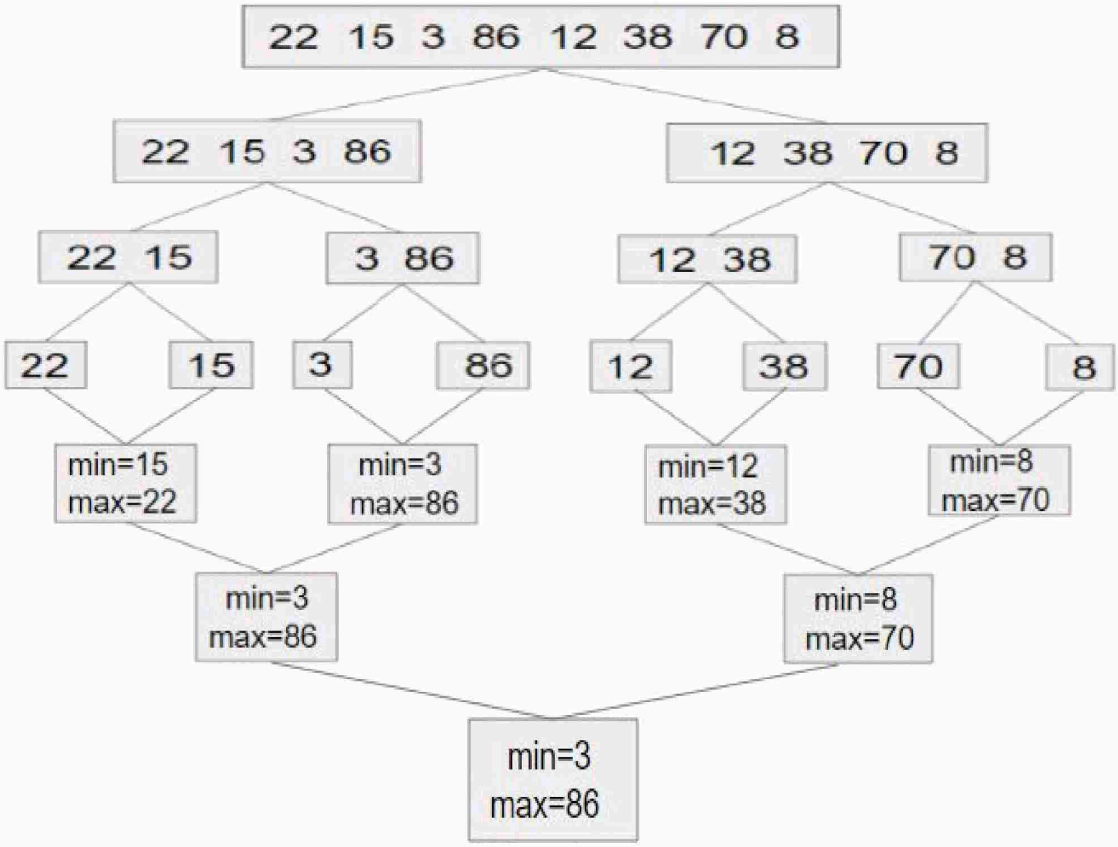
For finding minimum and maximum elements in the given list of elements follow the steps below:

1.we will divide the problem into sub-problems until they are small enough and combine the elements finding the minimum and maximum to form a group.

2. Of each group the max will compare with the only max of another group and min with min.

Eg :

Elements are 22,15,3,86,12,38,70,8. Find the minimum and maximum of the given elements.



Algorithm for finding minimum and maximum:

Algorithm minmaxDC(A[i,j])

//input : Array A with indices i and j.

//output : maximum and minimum elements of a given array.

begin

mid= $\lfloor \frac{i+j}{2} \rfloor$

[max1,min1]=minmaxDC(A[i,mid])

[max2,min2]=minmaxDC(A[mid+1,j])

return (min(min1,min2),max(max1,max2))

End

Time complexity:

$T(n)=2T(\frac{n}{2})+2$

Where n>2 a=2,b=2,d=0

$\therefore f(n) \varepsilon o(n^d)=o(1)$

And a> b^d by master's theorem

$T(n) \varepsilon o(n^{\log_b a})=o(n^{\log_2 2})$

$T(n) \varepsilon o(n)$

Merge Sort:

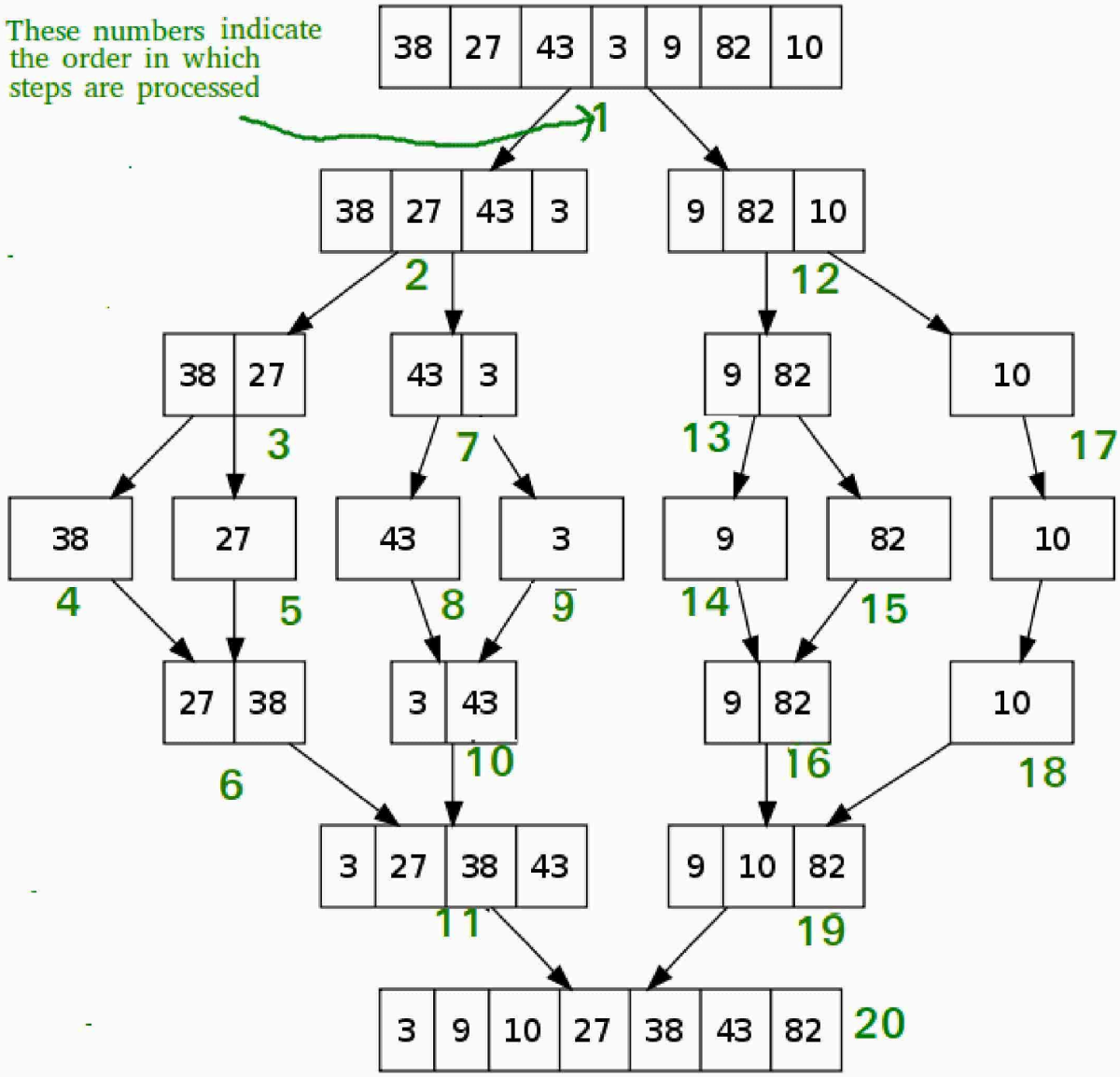
Merge sort is one of the most efficient sorting algorithms. It works on the principle of Divide and Conquer.

Eg : To perform merge sort on any list of elements given the steps below may help you:

- the array is recursively divided in two halves till the size becomes 1.
- Once the size becomes 1, the merge processes comes into action and starts merging arrays back till the complete array is merged.

Consider an array of elements a=[14, 7, 3, 12, 9, 11, 6, 2]

The merge sort of given array a is as shown below:



Algorithm for merge sort:

Algorithm mergesort(A[0...n-1]

//sort array A[0...n-1] by using a recursive approach.

//input : An array A[0...n-1]

//Output : An array A[0...n-1]

If n>1

copy A[0...($\frac{n}{2}$)-1] to B[0...($\frac{n}{2}$)-1]

copy A[($\frac{n}{2}$)...n-1] to C[0...($\frac{n}{2}$)-1]

mergesort B[0...($\frac{n}{2}$)-1]

mergesort C[0...($\frac{n}{2}$)-1]

merge(B,C,A)

merge(B[0...p-1],C[0...p-1],A[0...p+q-1])

//merges two sorted arrays into one.

//input : Array B[0...p-1] and C[0...q-1] both sorted.

//output : Sorted array A[0...p+q-1] of the elements.

i ← 0 , j ← 0 , k ← 0

while i<p and j<q do

if B[i] ≤ C[j]

A[k] ← C[j] ; i ← i+1

else

A[k] ← C[j] ; j ← j+1

k ← k+1

if i=p

copy C[j...q-1] to A[k...p+q-1]

else

copy B[i...p-1] to A[k...p+q-1]

Time complexity:

The merge sort is recursive algorithm and the time complexity of merge sort is as follows:

C(n)=2C($\frac{n}{2}$)+ C_{merge} (n)

for n>1,C[i]=0

C_{worst} (n)=2C($\frac{n}{2}$)+n-1

for n>1, C_{worst} (n)=0

$C_{worst}(n) \varepsilon o(n\log_2 n)$

Quick sort:

To perform quick sort follow the steps below:

1.Consider the first element as a pivot, the next element and last element positions be i and j.

2.While the value at the i in the array is less than the pivot value, add 1 to i. And continue until the value at the i is greater than or equal to the pivot value.

3.While the value at the j in the array is greater than the pivot value,subtract 1 from j. And continue until the value at the is less than or equal to the pivot value.

4.If the i is less than or equal to the j, then swap the values at these locations in the array.

5.If i is greater than j ,then swap the pivot and value at j.

6. When pivot is swapped it should be fixed and repeat the above steps until all the elements are fixed.

Eg :

Sort the list of elements given below by Quick sort.

9,7,5,11,12,2,14,3,10,6

Step-1:

9	7	5	11	12	14	3	10	6
0	1	2	3	4	5	6	7	8
p	i	i	i					j

i can be incremented until a[i] is less than 9 (p)

j can be decremented until a[j] is greater than 9(p)

Step-2: i<j so swap(a[i],a[j])

9	7	5	6	12	14	3	10	11
0	1	2	3	4	5	6	7	8
p	i	i	i	i		j	j	j

Step-3: i<j again swap(a[i],a[j])

9	7	5	6	3	14	12	10	11
0	1	2	3	4	5	6	7	8
p	i	i	i	i , j	j , i	j	j	j

Step-4: i>j so swap p,a[j]

3	7	5	6	9	14	12	10	11
0	1	2	3	4	5	6	7	8
				fixed				

Here pivot is fixed and quick sort is performed again on the two sub-arrays left and right to pivot.

Step-5:

3	7	5	6
0	1	2	3
p , j	i , j	j	j

14	12	10	11	
0	1	2	3	
p	i	i	j , i	i

Step-6:

In the 1st subarray i>j so swap a[j],p and p is fixed.

In the 2nd subarray i>j so swap p,a[j] and p is fixed.

3	7	5	6	
	0	1	2	
fixed	p	i	j , i	i

11	12	10	14
0	1	2	
p	i	j	fixed

Step-7:

In the 1st subarray i>j so swap a[j],p and p is fixed.

In the 2nd subarray i<j so swap a[i],a[j].

6	5		7
0	1		
p	i,j	i	fixed

11	10	12
0	1	2
p	i,j	j,i

step-8

In the 1st subarray i>j so swap a[j],p and p is fixed.

In the 2nd subarray i>j so swap p,a[j] and p is fixed.

5	6
0	1
fixed	fixed

10	11	12
0	1	2
fixed	fixed	fixed

All the elements in the subarrays are fixed. So the final sorted list is obtained combining all fixed elements at their positions

The sorted list is:

3	5	6	7	9	10	11	12	14
0	1	2	3	4	5	6	7	8

Algorithm for quick sort:

Algorithm Quicksort(a[l....r])

//sorts a subarray by quick sort.

//input: a subarray A[l...r] of A[0...n-1] defined by its left and right indices l and r

//output: The subarray A[l....r] sorted in ascending order.

If l<r

s ← partition(a[l...r]) //s is the split position

Quicksort(A[l...s-1])

Quicksort(A[s+1....r])

Algorithm partition(A[l...r])

//input: A subarray A[l...r] of A[0...n-1] defined by left and right indices l and r.

//output: A partition of A[l...r] with a split position is returned.

```
p ← A[ l ]
i ← l; j ← r+1
repeat
    repeat i ← i+1 until A[i]< p
    repeat j ← j-1 until A[j] > p
    swap(A[i],A[j])
until i ≥ j
swap(A[i],A[j])  //undo last swap
swap(A[l],A[j])
return j
```

Time complexity:

Worst case:

For comparing n ordered elements we require n+1 comparisions at each stage. For all the stages we require:

(n+1)+n+(n-1)+...+3 for n>1

(i.e., there must be atleast 2 elements for quicksort)

The above equation can be written as:

(n+1)+n+(n-1)+...+3+2+1-3

We know that

$n+(n-1)+(n-2)+.....+3+2+1= \frac{n(n+1)}{2}$

$\therefore C_{worst}(n) = \frac{(n+1)(n+2)}{2}$

$C_{worst}(n)\ \varepsilon\ o(n^2)$

Best case:

$C_{best}(n)$ =2C($\frac{n}{2}$)+n

For n>1

f(n) = n

n^d = n ⇒ d=1

a= b^d

∴ $C_{best}(n)$ =o(n^d log n)

=o(n logn)